Практическая работа 6. Типы памяти. Разделяемая память

В CUDA предусмотрено несколько типов памяти, каждый из которых предназначен под определенные нужды. Ниже приведена таблица типов памяти GPU.

Таблица 1

Типы памяти GPU

Тип памяти	**Доступ**	**Выделяется**	**Скорость работы**
Регистры	чтение/запись	на поток	Высокая
Локальная	чтение/запись	на поток	Низкая
Разделяемая	чтение/запись	на блок	Высокая
Глобальная	чтение/запись	на все решетки	Низкая
Константная	чтение	на все решетки	Низкая
Текстурная	чтение	на все решетки	Низкая

Регистры используется для хранения локальных переменных. Скорость доступа для данного типа памяти самая быстрая, и выделение происходит на каждую нить. **Локальная память** используется для хранения локальных переменных, когда регистров не хватает, скорость доступа низкая. Память выделяется так же, как и для регистров. **Разделяемая память** используется для хранения массивов данных, используемых совместно всеми нитями в блоке. Эта память имеет чуть меньшую скорость доступа, чем регистры, и выделяется на блок. В **глобальной памяти** хранятся исходно все данные, это общая память для всех блоков. **Константная память** используется для передачи параметров в ядро, превышающих допустимые размеры для параметров ядра, выделение целиком на решетку. **Текстурная память** используется для хранения больших массивов данных, выделяется целиком на решетку.

Обращение к памяти в CUDA осуществляется одновременно из половины количества нитей в блоке. Все типы памяти, кроме регистровой и локальной, можно использовать как эффективно, так и неэффективно. Поэтому необходимо четко понимать их особенности и области применения.

Рассмотрим более подробно разделяемую память. Основной особенностью разделяемой памяти, является то, что она оптимизирована под обращение к ней одновременно половины количества нитей в блоке (16 нитей). Вся разделяемая память разбита на 32 банка, состоящих из 32 битовых слов, причем, последовательно идущие слова попадают в последовательно идущие блоки. Обращение к каждому банку происходит независимо. Второй важной особенностью разделяемой памяти является то, что она выделяется на блок. Для задания разделяемой памяти используется спецификатор ***__shared__***. Часто использование разделяемой памяти требует явной синхронизации внутри блока.

При работе с разделяемой памятью необходимо обеспечить синхронизацию потоков, что в CUDA довольно просто – она предоставляет примитивный барьер, функцию __syncthreads(). При ее вызове потоки приостанавливаются до того момента, пока все потоки не достигнут этой точки. Важно, чтобы вызов __syncthreads() у всех потоков находился в одном и том же месте (в частности, был безусловным, т. е. не находился внутри оператора if и т.п.), в противном случае результаты могут быть непредсказуемыми, в частности, может возникнуть «мертвая блокировка» (deadlock).

Приведем фрагмент кода создания массива данных в разделяемой памяти.

```
#define BS 256 //Размер блока

__global__ void Kernel (float* data)
{
// Создание массива в разделяемой памяти
  __shared__ float a[BS];
  int i = blockIdx.x * BS + threadIDx.x;
// Копирование из глобальной памяти в разделяемую
  a[i] = data[i];
```

```
// Синхронизируем нити.
// Перед использованием нужно быть уверенным, что все
данные скопированились
  __syncthreads();
// Использонание данных разделяемой памяти
}
```

Данный пример демонстрирует только работу с разделяемой памятью. Для того, чтобы понять, как же разделяемая память помогает в ускорении работы, напишем программу для вычисления скалярного произведения векторов. Попробуем написать эту программу с использованием глобальной памяти. Используя механизм событий, замерим время работы ядра.

Рассмотрим, как изменится ядро, если вместо глобальной памяти использовать разделяемую память.

```
__global__ void scalMult(const BASE_TYPE *A, const
BASE_TYPE *B, BASE_TYPE *C, int numElem)
{
// Переменная для хранения суммы элементов
    BASE_TYPE sum = 0.0;
// Создание массивов в разделяемой памяти
  __shared__ BASE_TYPE ash[BLOCK_SIZE];
  __shared__ BASE_TYPE bsh[BLOCK_SIZE];
// Копирование из глобальной памяти
    ash[threadIdx.x] = A[blockIdx.x * blockDim.x +
    threadIdx.x];
    bsh[threadIdx.x] = B[blockIdx.x * blockDim.x +
    threadIdx.x];
// Синхронизация нитей
  __syncthreads();
// Вычисление скалярного произведения
  if (threadIdx.x == 0)
  {
    sum = 0.0;
    for (int j = 0; j < blockDim.x; j++)
      sum += ash[j] * bsh[j];
```

```
    C[blockIdx.x] = sum;
  }
}
```

Заменим ядро программы и оценим время работы ядра, использующего разделяемую память. Так как каждая нить теперь работает с памятью, чья скорость работы более высокая, большую часть времени работы программы занимают вычисления, а не доступ к памяти. Более наглядно это можно наблюдать в случае перемножения матриц.

Банки данных

Доступ к памяти осуществляется через 32 банка данных. Причем номер банка можно посчитать по формуле

Номер банка = (Адрес в байтах / 4) % 32.

Каждый банк в определенный момент времени обрабатывает одну транзакцию (чтение или запись). Оптимальный доступ к данным обеспечивается, когда каждая нить последовательно соотносится со своим банком (см. рис. 6а).

Также конфликта между банками нет, если каждой нити в любом порядке соответствует свой банк. Пример такого доступа изображен на рис. 6б. Конфликт имеет место, когда несколько нитей пытаются обратиться к одному банку данных, как изображено на рис. 6в. Однако в случае, когда все нити записывают или считывают с одного банка, конфликта не происходит.

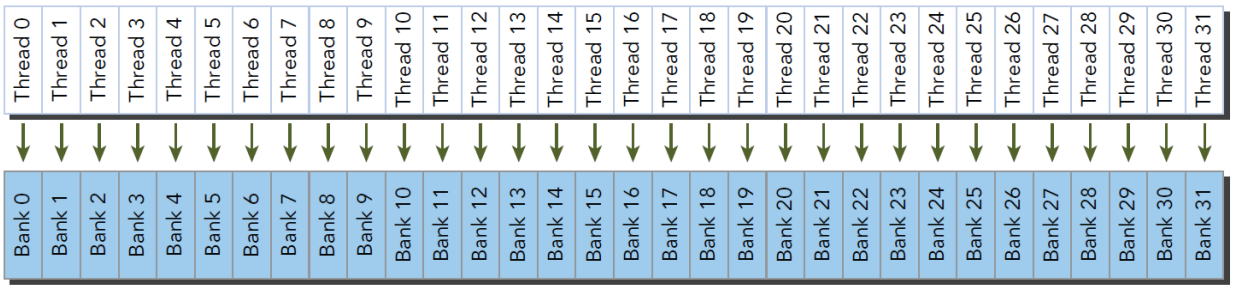


Рис. 6а. Оптимальный доступ к банкам данных

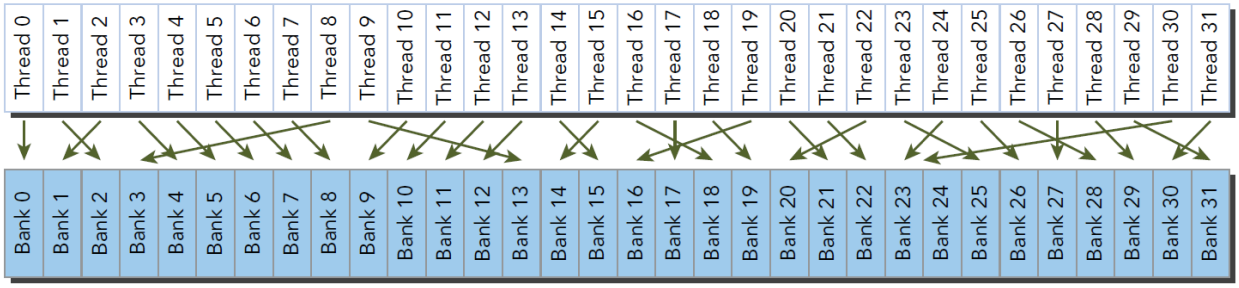


Рис. 6б. Бесконфликтный доступ к банкам данных

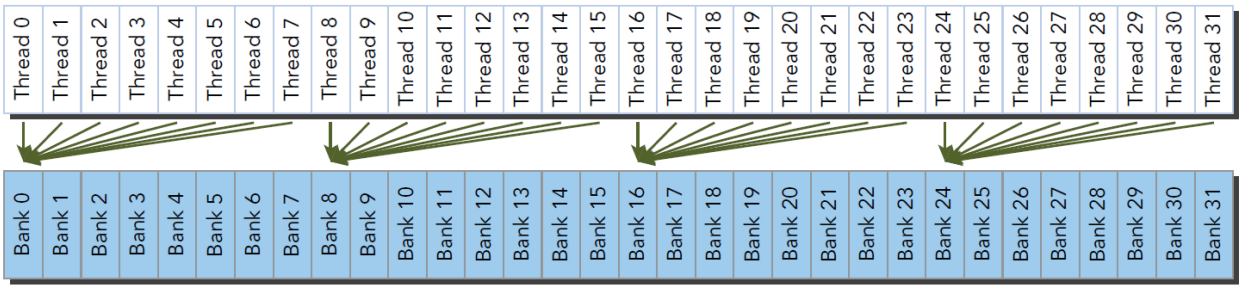


Рис. 6в. Конфликтный доступ к банкам данных

Рассмотрим наиболее распространенные случаи, когда могут иметь место конфликты банков данных. В следующем примере

```
__shared__ char s[N];
char data = s[threadIdx.x];
```

имеем однобайтовый массив, и, так как в одном банке содержится четыре байта, одновременно в один банк будут обращаться четыре нити. Это приведет к конфликту четвертого порядка (4 нити на 1 банк). Отметим, что на рис. 6в приведен пример конфликтов шестого порядка.

Один из способов решения такого конфликта состоит в добавлении избыточных данных:

```
__shared__ char s[4 * N];
char data = s[threadIdx.x * 4];
```

В следующем примере будет конфликт второго порядка (`short` – двухбайтовый тип)

```
__shared__ short s[N];
short data = s[threadIdx.x];
```

Чаще возникают конфликты, когда используются четырехбайтовые числа и номер в массиве вычисляется по формуле. Например, в следующем коде:

```
__shared__ float s[N];
float data = s[threadIdx.x * L];
```

при четных `L` возникают конфликты. Действительно, при `L` = 2 нулевая нить и 16-натая нить обращаются в нулевой банк, а при `L` = 4 уже нити с номерами 0, 8, 16 и 24 обращаются в нулевой банк, создавая конфликт четвертого порядка.

Пример. Оптимизация программы по перемножению двух матриц

Пусть даны две матрицы размером $N * N$ (для удобства будем считать, что N кратно 16). Если решать данную задачу, получая данные постоянно из глобальной памяти, то понадобится $2N$ чтений из глобальной памяти. Становится ясно, что основным тормозящим программу фактором является не что иное, как чтение из глобальной памяти, которая имеет низкую скорость передачи.

Заметно ускорить программу поможет правильная организация вычислений и разделяемая память. Обратим внимание на то, что для вычисления элемента новой матрицы необходимы лишь небольшие части исходных матриц. Для повышения быстродействия программы разобьем новую матрицу на подматрицы таким образом, чтобы обработкой данной подматрицы занимался только один блок. В данной задаче разобьем на подматрицы (C') размером 16*16 элементов (см. рис. 7).

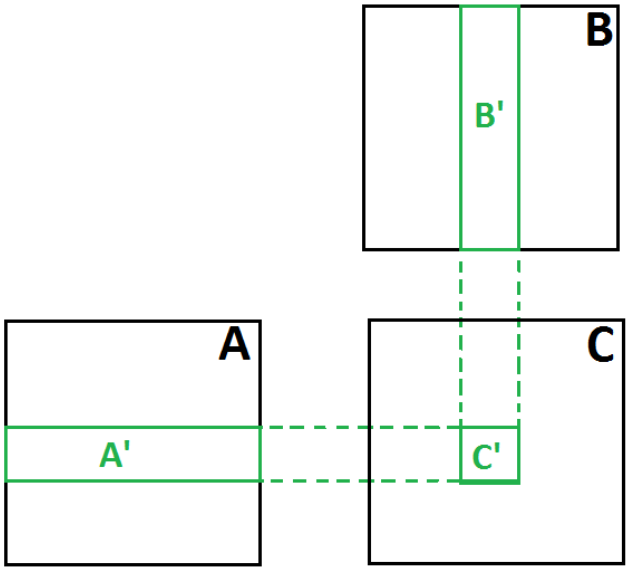


Рис. 7. Вычисление подматрицы новой матрицы

Однако, из-за малого объема разделяемой памяти, подматрицы A' и B' не могут быть целиком скопированы. Поступим с A' и B' так же, как поступили с подматрицей C', разобьем каждую матрицу на подматрицы размером 16*16. В итоге вычисление C' будем проводить в $N/16$ шагов (см. рис. 8).

Вычисление подматрицы C' можно записать следующим образом:

$$C' = A'_1 * B'_1 + A'_2 * B'_2 + \cdots + A'_{\frac{N}{16}} * B'_{\frac{N}{16}}.$$

Теперь на каждом шаге будем подгружать в разделяемую память по одной подматрице A'_i и B'_i, вычислять соответствующую им сумму для элементов произведения и по окончании всех вычислений записывать в результирующую матрицу.

Укажем в начале программы размер блока и тип элементов массива:

```
// размер блока или размер подматрицы
#define BLOCK_SIZE 16
//тип, который будут иметь элементы матриц
#define BASE_TYPE double
```

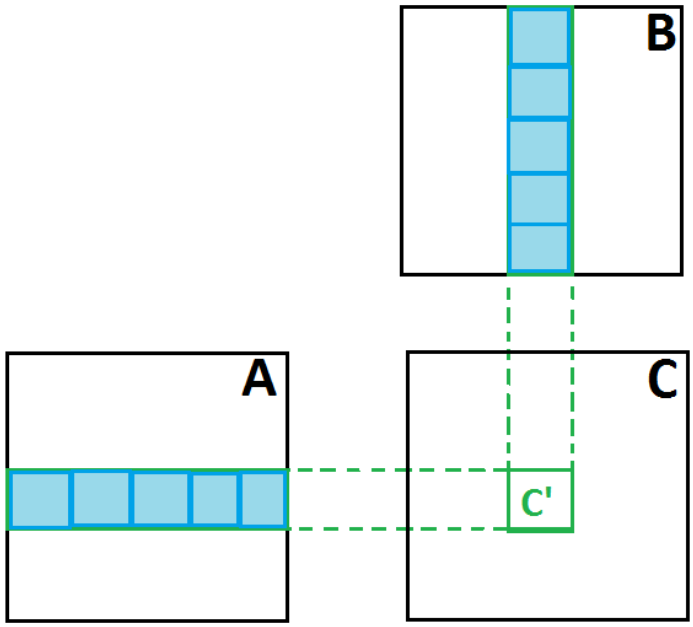


Рис. 8. Разбиение исходных матриц

С учетом всего вышесказанного об организации вычислений, ядро будет выглядеть следующим образом:

```
__global__ void matrixMult(const BASE_TYPE *A, const
 BASE_TYPE *B, BASE_TYPE *C, int Acols, int Bcols)
{
// индекс начала первой подматрицы A, которую
// обрабатывает блок
     int aBegin = Acols * blockDim.y * blockIdx.y;
 // индекс конца подматрицы A, которую обрабатывает блок
     int aEnd = aBegin + Acols - 1;
// шаг для перебора подматриц A
     int aStep = blockDim.x;
// индекс начала первой подматрицы B, которую
// обрабатывает блок
   int bBegin = blockDim.x * blockIdx.x;
// шаг для перебора подматриц B
     int bStep = blockDim.y * Bcols;

 // Выделение разделяемой памяти для подматриц
     __shared__ BASE_TYPE as [BLOCK_SIZE][BLOCK_SIZE];
     __shared__ BASE_TYPE bs [BLOCK_SIZE][BLOCK_SIZE];
```

```
// переменная для вычисления элемента подматрицы
    BASE_TYPE sum = 0.0;

    for (int ia = aBegin, ib = bBegin; ia < aEnd; ia +=
    aStep, ib += bStep)
    {
// загрузка подматриц A и B из глобальной памяти в
// разделяемую
        as[threadIdx.y][threadIdx.x] = A[ia + Acols *
        threadIdx.y + threadIdx.x];
        bs[threadIdx.y][threadIdx.x] = B[ib + Bcols *
        threadIdx.y + threadIdx.x];

// синхронизация нитей
      __syncthreads();

// перемножение двух матриц
      for (int k = 0; k < blockDim.x; k++)
            sum += as[threadIdx.y][k] *
            bs[k][threadIdx.x];

// синхронизация нитей
      __syncthreads();
    }

// индекс результирующего элемента в глобальной памяти
    int ind = Bcols * (blockDim.y * blockIdx.y +
    threadIdx.y) + blockDim.x * blockIdx.x + threadIdx.x;

// запись элемента в глобальную память
    C[ind] = sum;
}
```

После любой работы с разделяемой памятью необходимо синхронизировать нити. В данном примере важно произвести синхронизацию до момента начала расчетов, чтобы убедиться в том, что все данные загружены, а также после расчетов, чтобы убедиться в том, что текущие подматрицы не нужны и можно подгружать новые подматрицы.

Вызов ядра из главной функции:

```
// параметры запуска ядра
dim3 threadsPerBlock = dim3(BLOCK_SIZE, BLOCK_SIZE);
dim3 blocksPerGrid = dim3(Bcols / BLOCK_SIZE, Arows /
BLOCK_SIZE);

// старт времени работы ядра
cudaEventRecord( start, 0);

// вызов ядра
matrixMult<<<blocksPerGrid, threadsPerBlock>>>(d_A, d_B,
d_C, Acols, Bcols);

// стоп времени работы ядра
cudaEventRecord( stop, 0);
cudaEventSynchronize( stop );

// вычисление времени работы
float KernelTime;
cudaEventElapsedTime( &KernelTime, start, stop);
```

Использование такого алгоритма для перемножения двух матриц требует 2 * N / 16 чтений из глобальной памяти. Понятно, что правильное использование разделяемой памяти может поднять быстродействие программы даже более чем на порядок. Также метод вычисления суммы произведений элементов по блокам, который был применен в ядре для вычисления произведения двух матриц, ускоряет вычисления и часто применяется в подобных задачах.

Если размер разделяемой памяти не задан изначально и определяется в процессе работы программы, то необходимо его указывать при вызове ядра

```
myKernelFunc<<<gridSize, blockSize,
sharedMemSize>>>(float *param1, int *param2)
```

где `sharedMemSize` – размер дополнительной памяти, выделяемой при запуске ядра,

Практическая работа

1. Напишите программу для скалярного умножения двух векторов, используя разделяемую память. Разделяемая память выделяется с учетом ограничения ее размеров на блок. Количество нитей при вызове ядра равно числу элементов массива с разделяемой памятью. Замерьте время работы ядра программы.

2. Напишите программу вычисления определенного интеграла функции одной переменной по квадратурной формуле центральных прямоугольников.

3. Доработайте программу вычисления определенного интеграла, используя другие квадратурные формулы: формулы трапеций и Симпсона. Сравните точность результатов.

4. Напишите программу вычисления евклидовой нормы вектора (путем скалярного умножения его на себя и вычисления корня из произведения).

Контрольные вопросы

1. Какие типы памяти выделяют в технологии CUDA?

2. Какой спецификатор применяется для задания разделяемой памяти? Перечислите основные особенности разделяемой памяти.